05_pipeline/

Influence of Deep Tech-Based Digital Transformation on Quality and Efficiency of University Education in Emerging Contexts

5.1 Purpose

This pipeline operationalizes the technical artifact of the doctoral research by building a reproducible machine learning workflow to analyze how Deep Tech-based digital transformation influences university educational quality and institutional efficiency.

The pipeline uses a simple supervised classifier to predict whether a university shows a high or low level of digital transformation impact based on institutional, technological, pedagogical, and efficiency-related indicators.

5.2 Repository Structure

```
05_pipeline/
├── README.md
├── requirements.txt
├── Dockerfile
├── data/
│   └── university_digital_transformation.csv
├── src/
│   ├── train.py
│   ├── preprocess.py
│   └── utils.py
├── models/
│   └── classifier.pkl
├── mlruns/
├── dvc.yaml
├── dvc.lock
└── reports/
    └── metrics.json
```

5.3 Research-Oriented Prediction Task

The machine learning task is defined as follows:

Input variables

- AI adoption level
- Cloud computing use
- Learning analytics maturity
- Digital governance maturity
- Faculty digital competence
- Student digital engagement
- Administrative automation
- Resource optimization
- Institutional support
- Infrastructure readiness

Target variable

- Digital transformation impact
  - 1 = High impact
  - 0 = Low impact

This classification task supports the doctoral research by offering a reproducible analytical artifact for evaluating institutional patterns associated with Deep Tech-based transformation.

5.4 Reproducibility Stack

The pipeline follows the reproducibility stack proposed in Session 5.

Layer	**Tool**	**Function**
Code	Git / GitHub	Version control for scripts
Data	DVC	Dataset versioning
Experiments	MLflow	Parameter and metric tracking
Environment	Docker / requirements.txt	Dependency freezing
Reporting	README / metrics.json	Transparent documentation

5.5 Git Initialization

```
git init

git add .

git commit -m "Initial reproducible ML pipeline"
```

Git records every change in the project, allowing each experimental result to be connected to a specific code version.

5.6 DVC Data Versioning

```
pip install dvc

dvc init

dvc add data/university_digital_transformation.csv

git add data/university_digital_transformation.csv.dvc .gitignore

git commit -m "Track dataset with DVC"
```

DVC ensures that the dataset used for training is versioned and restorable, preventing silent data changes.

5.7 Seeded Training Script

The classifier applies the four-seed rule:

```
import os

import random

import numpy as np


SEED = 42


os.environ["PYTHONHASHSEED"] = str(SEED)

random.seed(SEED)

np.random.seed(SEED)
```

This makes the training process more repeatable, although Session 5 emphasizes that seeding is necessary but not sufficient for full reproducibility.

5.8 Simple Classifier

A baseline classifier such as Logistic Regression or Random Forest is used because the purpose of this artifact is reproducibility, not model complexity.

```
from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, f1_score
```

The pipeline prevents data leakage by splitting the data before fitting preprocessing transformations.

5.9 MLflow Experiment Tracking

```
import mlflow

import mlflow.sklearn


with mlflow.start_run():

    mlflow.log_param("model", "LogisticRegression")

    mlflow.log_param("seed", 42)

    mlflow.log_param("test_size", 0.20)


    mlflow.log_metric("accuracy", accuracy)

    mlflow.log_metric("f1_score", f1)


    mlflow.sklearn.log_model(model, "model")
```

MLflow creates a durable record of model parameters, metrics, and artifacts.

5.10 Environment File

```
pandas==2.2.2

numpy==1.26.4

scikit-learn==1.4.2
```

```
mlflow==2.12.1
dvc==3.50.1
joblib==1.4.2
```

Pinned dependencies reduce dependency drift and improve reproducibility.

5.11 Dockerfile

```
FROM python:3.11

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "src/train.py"]
```

Docker freezes the computational environment so that the pipeline can be executed on another machine.

5.12 Execution Commands

```
pip install -r requirements.txt
dvc pull
python src/train.py
mlflow ui
```

The user can then open the MLflow interface and inspect parameters, metrics, and model artifacts.

5.13 Stranger Test

A stranger should be able to reproduce the pipeline by running:

```
git clone <repository-url>
```

cd 05_pipeline

pip install -r requirements.txt

dvc pull

python src/train.py

mlflow ui

If the same dataset version, parameters, metrics, and model artifact are restored, the pipeline passes the reproducibility test.

5.14 Expected Output

The expected output includes:

- A trained baseline classifier.
- Accuracy and F1-score metrics.
- A saved model artifact.
- MLflow experiment logs.
- DVC-tracked dataset version.
- Git commit history.
- Reproducible execution instructions.

5.15 Scientific Relevance

This pipeline provides the technical foundation for the doctoral artifact by transforming the research topic into a reproducible analytical workflow. It supports transparency, auditability, and methodological rigor in the study of Deep Tech-based digital transformation in higher education.

Rather than presenting a single unverifiable model result, the pipeline ensures that each result can be traced to a specific code version, dataset version, parameter configuration, and computational environment.

5.16 Deliverable Summary

The final Session 5 deliverable is:

05_pipeline/

├── Git-versioned code

├── DVC-versioned dataset

├── Seeded classifier

├── MLflow experiment tracking

├── requirements.txt

├── Dockerfile

└── README.md

This artifact directly supports the doctoral research by providing a reproducible machine learning pipeline aligned with scientific integrity and research transparency.